NumPy — Complete Cheat Sheet

Coverage: arrays, creation, indexing, slicing, reshaping, dtypes, broadcasting, universal functions, statistics, sorting/searching, linear algebra, random, set operations, missing/NaN handling, file I/O, performance patterns, and commonly used methods.

Import: `import numpy as np`

1. Core Concepts & Array Creation

Create arrays

Concept / Syntax	Meaning / Usage
`np.array([1, 2, 3])`	1-D array
`np.array([[1,2],[3,4]])`	2-D array
`np.zeros((3,4))`	zeros
`np.ones((3,4))`	ones
`np.empty((2,3))`	uninitialized array
`np.full((2,3), 7)`	filled array
`np.eye(3)`	identity matrix
`np.identity(3)`	identity matrix
`np.diag([1,2,3])`	diagonal matrix
`np.arange(0, 10, 2)`	range with step
`np.linspace(0, 1, 5)`	evenly spaced values
`np.logspace(1, 3, 5)`	logarithmically spaced
`np.geomspace(1, 1000, 4)`	geometric spacing
`np.zeros_like(a), np.ones_like(a), np.full_like(a)`	match shape/dtype
`np.fromiter(iterable, dtype=int)`	build from iterator
`np.fromfunction(lambda i,j: i+j, (3,3))`	create from indices

Array attributes

Concept / Syntax	Meaning / Usage
`a.ndim`	number of dimensions
`a.shape`	dimensions
`a.size`	number of elements
`a.dtype`	data type
`a.itemsize`	bytes per element
`a.nbytes`	total bytes
`a.T`	transpose view
`a.base`	underlying array, useful for checking views

2. Data Types & Conversion

Common dtypes

Concept / Syntax	Meaning / Usage
`np.int8/int16/int32/int64`	
`np.uint8/uint16/uint32/uint64`	
`np.float16/float32/float64`	
`np.complex64/complex128`	
`np.bool_`	
`np.str_ / np.bytes_`	

Conversion & inspection

Concept / Syntax	Meaning / Usage
`a.astype(np.float64)`	convert dtype

Concept / Syntax	Meaning / Usage
`a.astype(int, copy=False)`	request no unnecessary copy
`a.item()`	extract scalar
`a.tolist()`	convert to Python list
`np.can_cast(from_, to_)`	test safe casting
`np.result_type(a, b)`	resulting dtype
`np.issubdtype(a.dtype, np.number)`	dtype category test

3. Indexing, Slicing & Selection

Basic indexing

Concept / Syntax	Meaning / Usage
`a[0], a[-1]`	
`a[1:5], a[:5], a[::2], a[::-1]`	
`a[1,2]`	row/column indexing
`a[:, 0]`	first column
`a[0, :]`	first row

Boolean & fancy indexing

Concept / Syntax	Meaning / Usage
`a[a > 10]`	boolean filtering
`a[(a > 10) & (a < 20)]`	combined conditions; use & | ~, not and/or
`a[[0,2,4]]`	select specific indices
`a[:, [0,2]]`	select columns
`np.where(a > 0, 1, 0)`	conditional selection
`np.nonzero(a)`	indices of non-zero values
`np.argwhere(a > 0)`	coordinates matching condition
`np.flatnonzero(a)`	flattened non-zero indices

Conditional helpers

Concept / Syntax	Meaning / Usage
`np.select([cond1, cond2], [x, y], default=z)`	
`np.clip(a, min, max)`	bound values
`np.take(a, indices, axis=...)`	indexed selection

4. Reshaping, Combining & Splitting

Shape operations

Concept / Syntax	Meaning / Usage
`a.reshape(2,3)`	reshape
`a.reshape(-1, 1)`	infer one dimension
`a.ravel()`	flattened view when possible
`a.flatten()`	flattened copy
`a.squeeze()`	remove dimensions of size 1
`np.expand_dims(a, axis=0)`	add dimension
`a[..., np.newaxis]`	add dimension
`a.swapaxes(0,1)`	swap axes
`a.transpose() / np.transpose(a)`	transpose
`np.moveaxis(a, source, destination)`	move axis

Join arrays

Concept / Syntax	Meaning / Usage
`np.concatenate([a,b], axis=0)`	

Concept / Syntax	Meaning / Usage
`np.stack([a,b], axis=0)`	add a new axis
`np.vstack([a,b])`	vertical stack
`np.hstack([a,b])`	horizontal stack
`np.dstack([a,b])`	depth stack
`np.column_stack([a,b])`	columns
`np.block([[a,b],[c,d]])`	block composition

Split arrays

Concept / Syntax	Meaning / Usage
`np.split(a, 2, axis=0)`	
`np.array_split(a, 3, axis=0)`	allows uneven splits
`np.vsplit(a, 2)`	
`np.hsplit(a, 2)`	
`np.dsplit(a, 2)`	

5. Broadcasting

Rules

Concept / Syntax	Meaning / Usage
`Compare dimensions from right to left.`	
`Two dimensions are compatible when they are equal or one is 1.`	
`Missing leading dimensions are treated as 1.`	
`Example: (3,1) + (1,4) -> (3,4).`	

Useful patterns

Concept / Syntax	Meaning / Usage
`a + scalar`	scalar broadcasts
`a + vector`	vector broadcasts across compatible axis
`a[:, None]`	turn (n,) into (n,1)
`a[None, :]`	turn (n,) into (1,n)
`np.broadcast_to(a, shape)`	explicit broadcast view
`np.broadcast_arrays(a,b)`	broadcast multiple arrays

6. Universal Functions (ufuncs)

Arithmetic

Concept / Syntax	Meaning / Usage
`np.add(a,b), np.subtract(a,b), np.multiply(a,b), np.divide(a,b)`	
`np.floor_divide(a,b), np.mod(a,b), np.power(a,b)`	
`np.negative(a), np.absolute(a), np.sign(a)`	
`np.reciprocal(a)`	

Exponential & logarithmic

Concept / Syntax	Meaning / Usage
`np.exp(a), np.expm1(a)`	
`np.log(a), np.log1p(a), np.log2(a), np.log10(a)`	

Rounding

Concept / Syntax	Meaning / Usage
`np.round(a, decimals=2), np.around(a)`	
`np.floor(a), np.ceil(a), np.trunc(a)`	

Trigonometry

Concept / Syntax	Meaning / Usage
`np.sin, np.cos, np.tan`	
`np.arcsin, np.arccos, np.arctan, np.arctan2`	
`np.degrees, np.radians`	
`np.sinh, np.cosh, np.tanh`	

Comparison & logical

Concept / Syntax	Meaning / Usage
`np.equal, np.not_equal, np.greater, np.greater_equal, np.less, np.less_equal`	
`np.logical_and, np.logical_or, np.logical_not, np.logical_xor`	
`np.maximum(a,b), np.minimum(a,b)`	
`np.fmax(a,b), np.fmin(a,b)`	NaN-aware pairwise min/max

7. Aggregation & Statistics

Core aggregations

Concept / Syntax	Meaning / Usage
`np.sum(a, axis=...), np.prod(a, axis=...)`	
`np.min(a, axis=...), np.max(a, axis=...)`	
`np.mean(a, axis=...), np.median(a, axis=...)`	
`np.std(a, axis=...), np.var(a, axis=...)`	
`np.ptp(a, axis=...)`	max-min range
`np.percentile(a, q), np.quantile(a, q)`	
`np.average(a, weights=...)`	weighted mean

NaN-aware versions

Concept / Syntax	Meaning / Usage
`np.nansum, np.nanprod, np.nanmin, np.nanmax`	
`np.nanmean, np.nanmedian, np.nanstd, np.nanvar`	
`np.nanpercentile, np.nanquantile`	

Correlation & covariance

Concept / Syntax	Meaning / Usage
`np.corrcoef(x, y)`	
`np.cov(x, y)`	

Cumulative operations

Concept / Syntax	Meaning / Usage
`np.cumsum(a, axis=...), np.cumprod(a, axis=...)`	
`np.nancumsum(a), np.nancumprod(a)`	

8. Sorting, Searching & Counting

Sorting

Concept / Syntax	Meaning / Usage
`np.sort(a, axis=-1)`	sorted copy
`a.sort(axis=-1)`	in-place
`np.argsort(a)`	indices that sort array
`np.lexsort((secondary, primary))`	multi-key sorting
`np.partition(a, kth)`	partial sort
`np.argpartition(a, kth)`	indices for partial sort

Search

Concept / Syntax	Meaning / Usage
`np.argmax(a), np.argmin(a)`	
`np.nanargmax(a), np.nanargmin(a)`	
`np.searchsorted(sorted_a, values)`	
`np.where(condition)`	
`np.flatnonzero(condition)`	

Counting

Concept / Syntax	Meaning / Usage
`np.count_nonzero(a)`	
`np.unique(a, return_counts=True)`	
`np.bincount(integer_array)`	

9. Set Operations

Functions

Concept / Syntax	Meaning / Usage
`np.unique(a)`	
`np.union1d(a,b)`	
`np.intersect1d(a,b)`	
`np.setdiff1d(a,b)`	
`np.setxor1d(a,b)`	
`np.isin(a, values)`	

10. Linear Algebra

Matrix operations

Concept / Syntax	Meaning / Usage
`a @ b`	matrix multiplication
`np.matmul(a,b), np.dot(a,b)`	
`np.inner(a,b), np.outer(a,b)`	
`np.kron(a,b)`	Kronecker product
`np.linalg.matrix_transpose(a)`	

Decomposition & solving

Concept / Syntax	Meaning / Usage
`np.linalg.inv(A)`	inverse; avoid when solve() is possible
`np.linalg.solve(A,b)`	solve Ax=b
`np.linalg.lstsq(A,b)`	least-squares solution
`np.linalg.det(A)`	determinant
`np.linalg.matrix_rank(A)`	rank
`np.linalg.eig(A)`	eigenvalues/eigenvectors
`np.linalg.eigh(A)`	symmetric/Hermitian eigenproblem
`np.linalg.svd(A)`	singular value decomposition
`np.linalg.qr(A)`	QR decomposition
`np.linalg.cholesky(A)`	Cholesky decomposition

Norms & metrics

Concept / Syntax	Meaning / Usage
`np.linalg.norm(a)`	
`np.linalg.vector_norm(a)`	
`np.linalg.matrix_norm(A)`	

Concept / Syntax	Meaning / Usage
`np.linalg.cond(A)`	condition number

11. Random Number Generation

Recommended Generator API

Concept / Syntax	Meaning / Usage
`rng = np.random.default_rng(seed=42)`	
`rng.random((3,3))`	uniform [0,1)
`rng.integers(0, 10, size=(3,3))`	
`rng.uniform(0, 1, size=10)`	
`rng.normal(loc=0, scale=1, size=10)`	
`rng.standard_normal(size=10)`	
`rng.binomial(n=10, p=0.5, size=10)`	
`rng.poisson(lam=3, size=10)`	
`rng.exponential(scale=2, size=10)`	
`rng.choice(a, size=5, replace=False)`	
`rng.permutation(a)`	shuffled copy
`rng.shuffle(a)`	shuffle in place

Legacy API (know it, but prefer Generator)

Concept / Syntax	Meaning / Usage
`np.random.seed(42)`	
`np.random.rand(...), np.random.randn(...)`	
`np.random.randint(...), np.random.choice(...)`	
`np.random.shuffle(...),` `np.random.permutation(...)`	

12. Missing Values, NaN & Infinity

Detection

Concept / Syntax	Meaning / Usage
`np.isnan(a)`	
`np.isinf(a)`	
`np.isfinite(a)`	
`np.isneginf(a), np.isposinf(a)`	

Replacement / handling

Concept / Syntax	Meaning / Usage
`np.nan_to_num(a, nan=0, posinf=..., neginf=...)`	
`mask = np.isnan(a); a[mask] = value`	
`np.ma.masked_invalid(a)`	masked array for invalid values

13. Copy vs View

Key methods

Concept / Syntax	Meaning / Usage
`b = a`	same array object
`b = a.view()`	view; shares data
`b = a.copy()`	independent copy
`np.shares_memory(a,b)`	checks actual memory sharing
`np.may_share_memory(a,b)`	conservative memory-sharing check

Rule of thumb

Concept / Syntax	Meaning / Usage
`Slicing often returns a view.`	
`Fancy indexing and boolean indexing generally return copies.`	
`Use copy() when you need independent data.`	

14. File I/O

Binary NumPy formats

Concept / Syntax	Meaning / Usage
`np.save('data.npy', a)`	
`a = np.load('data.npy')`	
`np.savez('data.npz', x=a, y=b)`	
`np.savez_compressed('data.npz', x=a)`	

Text / CSV

Concept / Syntax	Meaning / Usage
`np.savetxt('data.csv', a, delimiter=',', header='x,y', comments='')`	
`a = np.loadtxt('data.csv', delimiter=',')`	
`a = np.genfromtxt('data.csv', delimiter=',', names=True, filling_values=np.nan)`	

15. Polynomial & Mathematical Helpers

Useful functions

Concept / Syntax	Meaning / Usage
`np.polyval(p, x)`	evaluate polynomial
`np.polyfit(x, y, deg)`	least-squares polynomial fit
`np.roots(p)`	polynomial roots
`np.diff(a)`	discrete differences
`np.gradient(a)`	numerical gradient
`np.trapezoid(y, x)`	trapezoidal integration
`np.cumsum(a), np.cumprod(a)`	
`np.gcd(a,b), np.lcm(a,b)`	

16. Date/Time Arrays

Basics

Concept / Syntax	Meaning / Usage
`np.datetime64('2026-08-12')`	
`np.timedelta64(7, 'D')`	
`date + np.timedelta64(1, 'D')`	
`np.isnat(a)`	missing datetime/timedelta

17. String Operations

Vectorized string functions

Concept / Syntax	Meaning / Usage
`np.char.add(a,b)`	
`np.char.lower(a), np.char.upper(a)`	
`np.char.strip(a)`	
`np.char.replace(a, old, new)`	
`np.char.startswith(a, prefix), np.char.endswith(a, suffix)`	

Concept / Syntax	Meaning / Usage
`np.char.find(a, sub)`	

18. Advanced / Useful Utilities

Iteration & conversion

Concept / Syntax	Meaning / Usage
`np.nditer(a)`	iterator
`np.ndindex(shape)`	index iterator
`np.ndenumerate(a)`	index + value iterator
`np.array_equal(a,b)`	exact equality
`np.array_equiv(a,b)`	shape-compatible equality
`np.allclose(a,b, rtol=..., atol=...)`	floating-point comparison

Memory / shape helpers

Concept / Syntax	Meaning / Usage
`np.atleast_1d(a), np.atleast_2d(a), np.atleast_3d(a)`	
`np.broadcast_to(a, shape)`	
`np.tile(a, reps)`	repeat whole pattern
`np.repeat(a, repeats, axis=...)`	repeat elements
`np.pad(a, pad_width, mode=...)`	

Window / grid helpers

Concept / Syntax	Meaning / Usage
`np.meshgrid(x, y)`	
`np.indices(dimensions)`	
`np.triu(A), np.tril(A)`	
`np.diagflat(a)`	

19. Axis — The Most Important Concept

Mental model

Concept / Syntax	Meaning / Usage
`axis=0 operates down rows / across the first dimension; for a 2-D array it reduces rows and returns one value per column.`	
`axis=1 operates across columns; for a 2-D array it reduces columns and returns one value per row.`	
`axis=None reduces the entire array.`	
`Example: a.shape == (3,4): np.sum(a, axis=0) -> shape (4,), np.sum(a, axis=1) -> shape (3,).`	

20. Performance Patterns for Data Engineering

Prefer vectorization

Concept / Syntax	Meaning / Usage
`Prefer a + 10 over Python loops.`	
`Prefer boolean masks over repeated if statements.`	
`Use broadcasting instead of manually constructing repeated arrays.`	
`Use ufuncs and reductions for numerical workloads.`	

Avoid unnecessary copies

Concept / Syntax	Meaning / Usage
Use views where safe.	
Use dtype appropriate to the data.	
Use np.load(..., mmap_mode='r') for large .npy files when appropriate.	
Chunk very large datasets rather than loading everything into RAM.	

NumPy vs pandas

Concept / Syntax	Meaning / Usage
NumPy: homogeneous numerical arrays, vectorized numerical computation, linear algebra.	
pandas: labeled tabular data, mixed dtypes, missing-data workflows, joins/grouping.	
Typical DE workflow: NumPy for numerical kernels; pandas/Polars/Spark for tabular processing depending on scale.	

21. Quick Method Map

Array methods / attributes to memorize

Concept / Syntax	Meaning / Usage
reshape, ravel, flatten, squeeze, transpose, swapaxes	
astype, copy, view, sort, argsort	
sum, mean, median, std, var, min, max, prod	
cumsum, cumprod, diff	
any, all	

Top np.* functions to memorize

Concept / Syntax	Meaning / Usage
array, zeros, ones, full, arange, linspace	
concatenate, stack, vstack, hstack, split	
where, select, unique, isin	
sort, argsort, argmax, argmin	
mean, median, std, var, percentile, quantile	
nanmean, nansum, nanmedian	
save, load, savetxt, genfromtxt	
linalg.solve, inv, eig, svd, norm	
random.default_rng	

22. Common Mistakes

Avoid these

Concept / Syntax	Meaning / Usage
Using Python and/or with NumPy arrays	use &, |, ~ with parentheses.
Confusing axis=0 and axis=1.	
Using np.matrix	prefer regular ndarray and @.
Using np.random.seed globally in new code	prefer np.random.default_rng().
Comparing floats with ==	use np.isclose/allclose.
Assuming reshape always copies	it may return a view.
Using np.append repeatedly in a loop	accumulate and concatenate once.
Using np.vectorize expecting true numerical vectorization	it mainly provides convenience, not speed.
Using np.linalg.inv(A) @ b when np.linalg.solve(A,b) is available.	